

Reservoir Architecture

Bigram encoder and double-delay reservoir

This model represents a simple recurrent reservoir fed by a bigram encoder. A readout node (predictive hetero-associative memory component) learns to predict the next input based on the current input and the state of the reservoir.

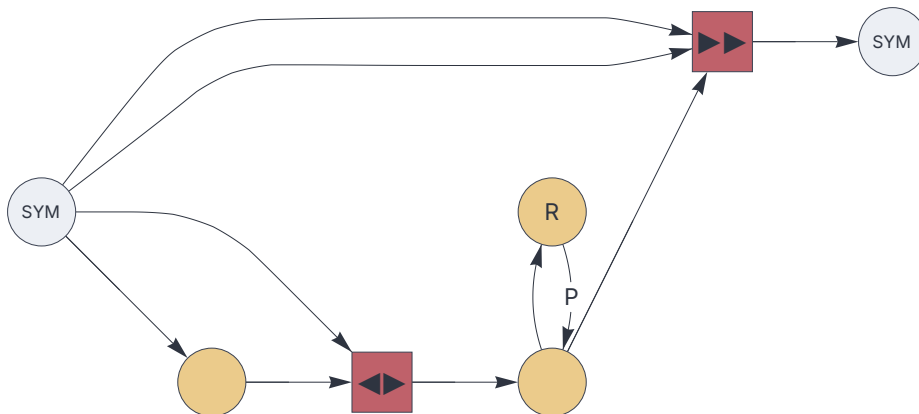
This architecture functions as a basic sequence memory.

Configuration

```
<< "Circuits.m"
```

```
dataflow = {  
    input[codec] [11] [],  
    delay[] [21] [11],  
    heteroencoder[] [51] [11, 21],  
    delay[] [61] [{51, 62 "P"}],  
    delay["R" → 30] [62] [61],  
    predictor[] [101] [11, {11, 61}],  
    output[codec] [] [101],  
    _ → {2000, 8}  
};
```

```
c = Circuit[dataflow]; c[Graph, ImageSize→ 500]
```



Sequence learning

```
{Null, Null, p, Null, e, Null, p, {p, l}, e, A,  
p, p, l, e, A, p, p, l, e, A, p, p, l, e, A, p, p, l, e, A}
```

Random sequence data

A sequence of 1000 digits:

```
dataset = RandomInteger[ {1, 10}, 1000];
```

Analyze prediction score

```
Do[  
  predictions = c /@ dataset;  
  Print["  iteration = ", i,  
    ", prediction score = ", N[Mean@Boole@MapThread[SameQ,  
      {dataset, RotateRight[predictions]}]]],  
  {i, 1, 5}];
```

```
iteration = 1, prediction score = 0.096
```

```
iteration = 2, prediction score = 0.993
```

```
iteration = 3, prediction score = 1.
```

```
iteration = 4, prediction score = 1.
```

```
iteration = 5, prediction score = 1.
```